How to Evaluate a Model (Part 1)

DSC 340

Week 4, Day 2, Feb. 28

The Plan

- Train-test process
- Accuracy score
- Sampling methods
- Data leakage
- Cross-validation
- (if time) Extended example: Jupyter Notebook
- (next class) Better metrics and relative cost of errors

Train-test process

1. Randomly **partition** data into **training** and **test** sets (**train-test split**)
2. **Initialize** classifier
3. Use **training set** to **fit** a model (**learn model parameters**)
4. Use learned model to **predict targets** on **test** set
5. **Evaluate** the model with various **metrics**

TL;DR: We are conducting a scientific experiment.

Q.: What if we want to try more than one model?

Cross-validation

- Conduct experiment within experiment
- Further partition training data into **training** and **validation** set
- Train each model on training set
- Use metrics to compare performance on validation set
- Pick the best performing model
- All else equal, we prefer the simplest model (**Occam's razor**)
- Finally, evaluate best model on **test** (**holdout**) set
- Important: Don't touch test set until the very end! (else **data leakage**)

Train-test split

Evaluate the model on a random sample

Evaluating a model

- How well does it generalize?
- **Random sample of data**
- Why? We have Big Data!
- **Labeled data** often scarce

Train-test split

- Partition labeled data: **training** and **testing**
- **Learn model** (e.g., $\vec{w}, w_0$) from **training** set
- Use model to **predict** $\hat{y}$ from $\vec{x}$ on **test** set (assume **labeled**)
- **Compare predicted** $\hat{y}$ **to true** y for each example

Train-test split



https://elitedatascience.com/overfitting-in-machine-learning#how-to-detect

Accuracy score and errors

- Success: prediction correct $y_i = \hat{y}_i$
- Failure: prediction incorrect $y_i \neq \hat{y}_i$
- **Accuracy score** (success rate)

$$accuracy\ score = \frac{\#\ correct}{\#\ correct\ +\ \#\ incorrect}$$

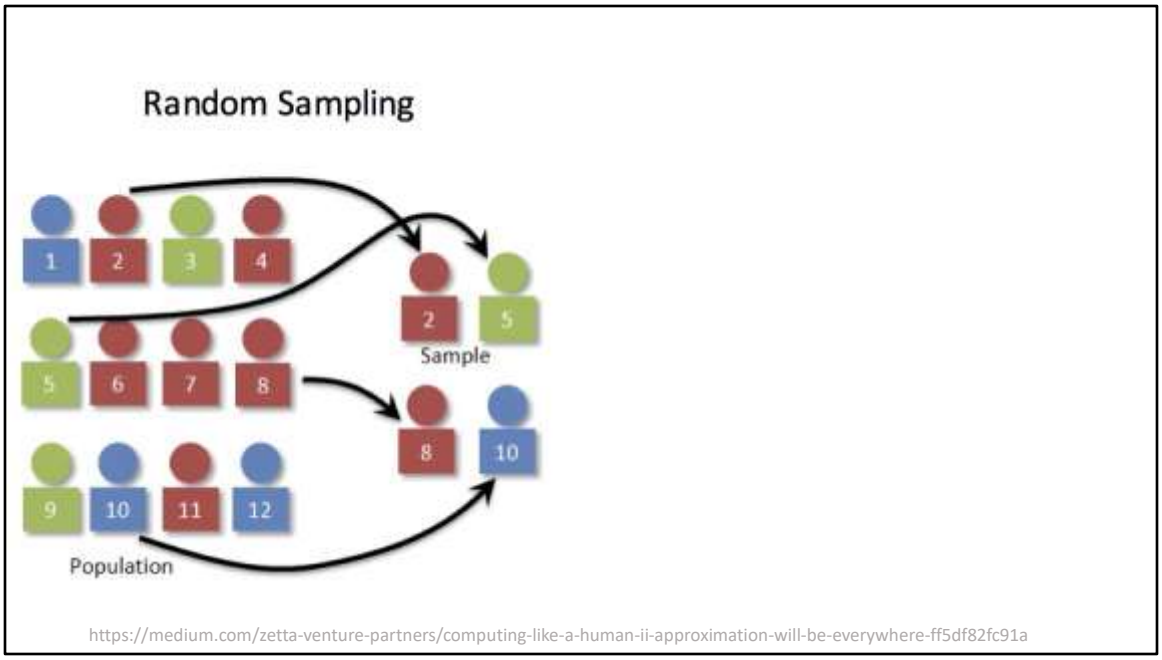
- **Error rate** proportion incorrect $1 - accuracy\ score$
- Bias error fail to learn training set underfitting
- Variance doesn't generalize to test set overfitting

Sampling

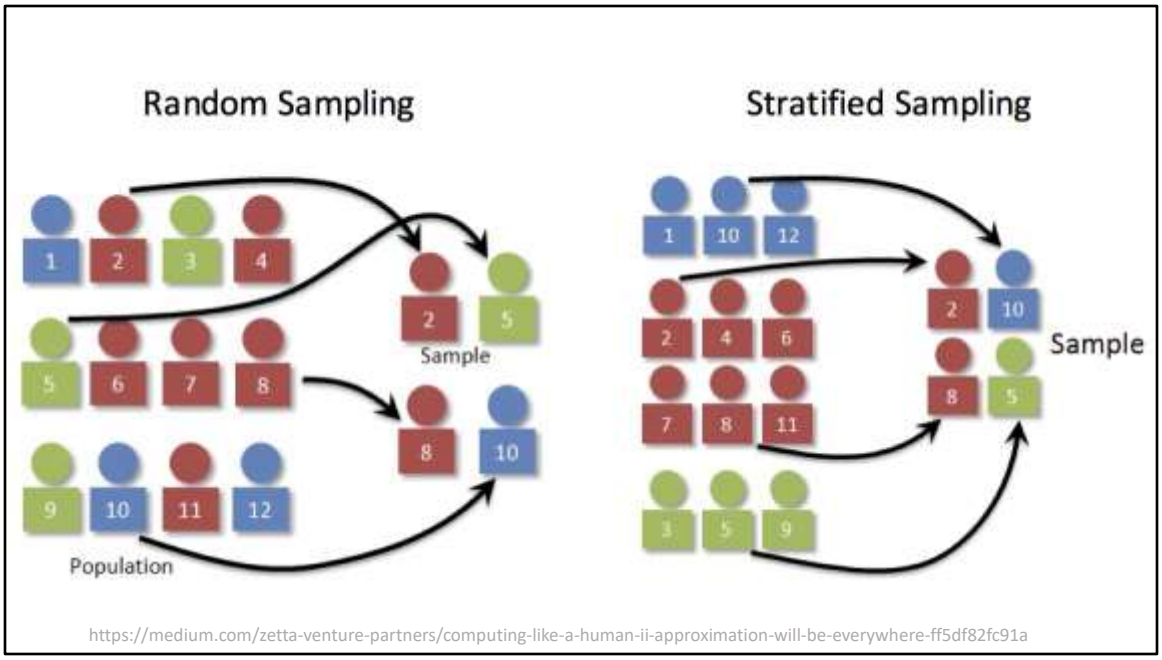
- Independent, representative samples (**i.i.d.**)
- **i.i.d.** = **independent** and **identically distributed**
- No selection depends on any other selection
- Same probability distribution for each selection
- Common statistical assumption

Sampling (cont.)

- **Representative** sample
- How do we know?
- Simple test: is each class represented?
- Example 1: Suppose survey only includes students in Greek life
- Example 2: What if non-binary persons don't occur in survey?
- **Stratified sampling**
- Strata is problem-specific



https://medium.com/zetta-venture-partners/computing-like-a-human-ii-approximation-will-be-everywhere-ff5df82fc91a



https://medium.com/zetta-venture-partners/computing-like-a-human-ii-approximation-will-be-everywhere-ff5df82fc91a

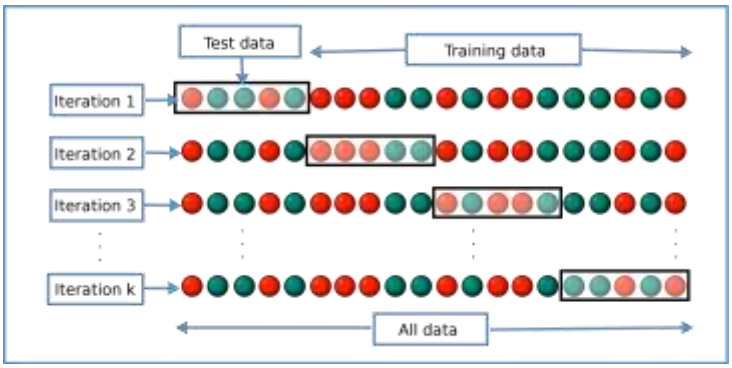
Data Leakage

- Don’t touch holdout (test) set until the end!
- If we let it influence our choice of models, that is data leakage.
- **Don’t:** Run lots of trials, evaluating on test set each time
- **Don’t:** Different teams explore same data set repeatedly
- **Don’t:** Preprocess training and test sets together
- Rule #1: Don’t touch the **test (holdout)** data
- Rule #2: See rule #1
- Important: Fit scaling parameters to train: then transform both sets

Cross-Validation

Section 5.3

Cross Validation (k-fold C.V.)



Special case: $k = N$

Known as **Leave-One-Out Cross Validation (LOOCV)**

Cross-Validation

- *k*-folds
  - Single holdout: *k* = 1
- Training/Testing
- Examples:
  - Threefold cross-validation
  - Stratified 10-fold cross-validation
- Still random variation …
- *Repeated* k-fold cross-validation

Extended Example

See Jupyter Notebook how_to_evaluate.ipynb